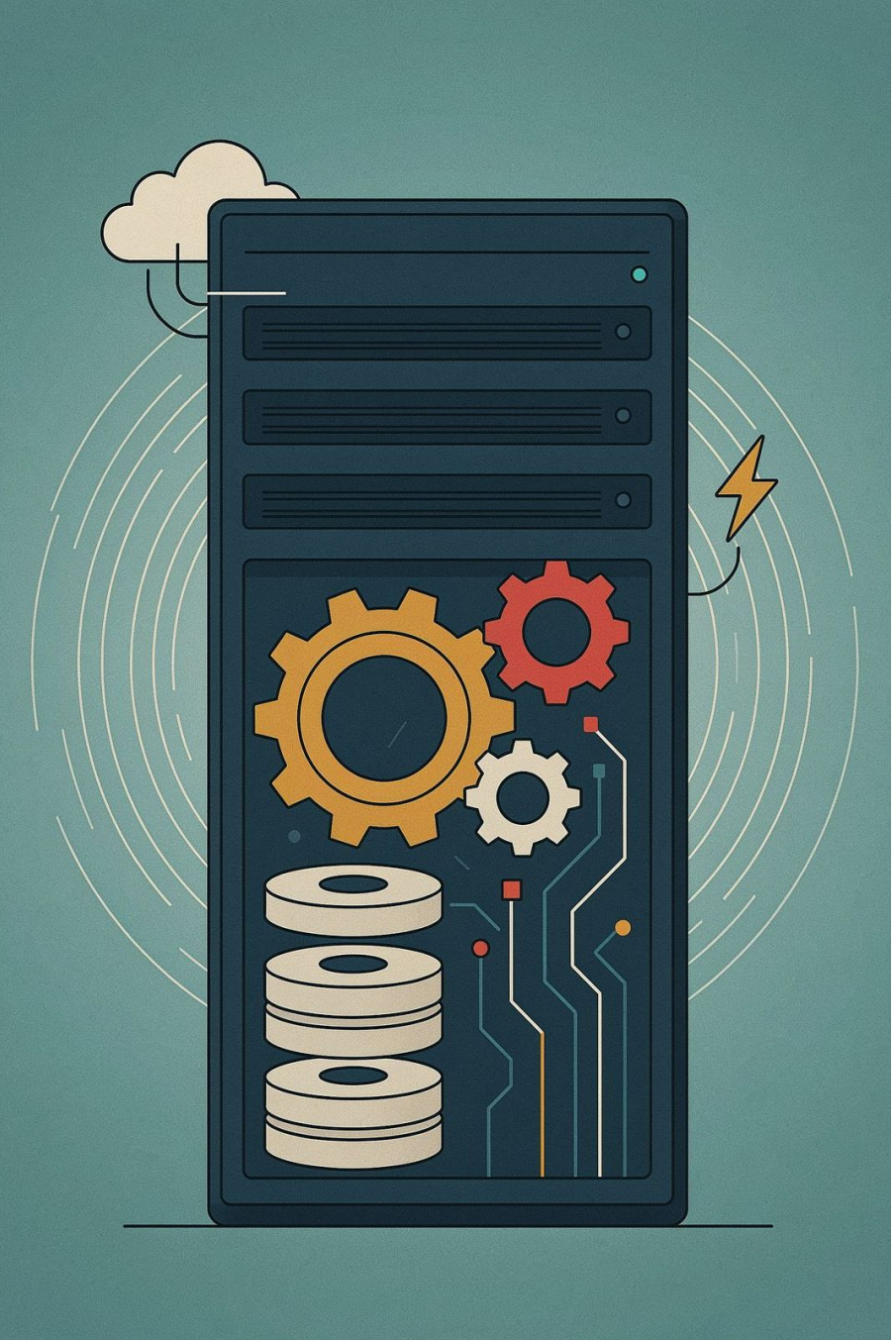
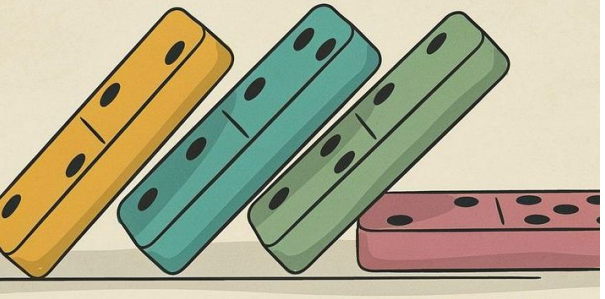


Triggers no SQLite

Automatizando Regras de Negócio com Gatilhos de Banco de Dados

Aprenda a usar triggers para manter a integridade dos dados, automatizar cálculos e garantir consistência em seu banco de dados SQLite — sem escrever uma linha extra de código na aplicação.



O que é uma Trigger?

Uma **trigger** (ou gatilho) é um bloco de código SQL executado automaticamente pelo banco de dados em resposta a um evento específico — como uma inserção, atualização ou exclusão de registro. Pense nela como uma regra automática que "dispara" sempre que determinada ação ocorre, como por exemplo:

AFTER INSERT

Executa após a inserção de um novo registro na tabela monitorada.

BEFORE UPDATE

Executa antes que uma atualização seja aplicada, permitindo validação ou transformação dos dados.

AFTER DELETE

Executa após a exclusão de um registro, útil para auditorias ou limpeza de dados relacionados.

Triggers são executadas automaticamente pelo banco de dados, garantindo que as regras de negócio sejam sempre aplicadas, independentemente de qual aplicação faz a operação.

Por que Usar Triggers?

Triggers resolvem um problema clássico: garantir que regras de negócio sejam aplicadas de forma centralizada e consistente, sem depender de cada aplicação ou desenvolvedor lembrar de executar as verificações corretas.



Integridade dos Dados

Evitam estados inconsistentes, garantindo que valores derivados — como totais e saldos — estejam sempre atualizados automaticamente.



Automação de Regras

Eliminam código repetitivo na camada de aplicação, centralizando a lógica de negócio diretamente no banco de dados.



Auditoria e Rastreamento

Permitem registrar automaticamente quem alterou o quê e quando, criando um histórico completo de modificações.



Validação Centralizada

Aplicam restrições de negócio antes ou depois de qualquer operação, independentemente da fonte da requisição.

O Cenário Prático: Sistema de Ordem de Serviço

Vamos trabalhar com um exemplo real e concreto: um sistema de **Ordem de Serviço**, onde cada ordem pode conter múltiplos serviços. O desafio é manter o campo `valor_total` da ordem sempre atualizado conforme novos serviços são adicionados.

Tabela: `ordem`

Tabela principal que representa cada Ordem de Serviço aberta para um cliente.

- `id` — Identificador único da ordem
- `valor_total` — **Soma acumulada** de todos os serviços
- `data_criacao` — Data de abertura da ordem

Tabela: `ordem_servico`

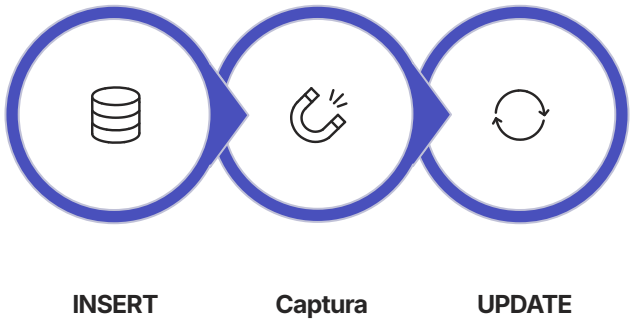
Tabela intermediária que registra cada serviço vinculado a uma ordem específica.

- `id` — Identificador do registro
- `id_ordem` — Chave estrangeira para `ordem`
- `preco` — Valor unitário do serviço
- `quantidade` — Quantidade de vezes executada

A Lógica do Negócio: Calculando o Acréscimo

Sempre que um novo serviço for inserido na tabela `ordem_servico`, o campo `valor_total` da ordem correspondente deve ser incrementado automaticamente. A fórmula é simples e direta:

$$valor_total = valor_total + (preco \times quantidade)$$



O banco de dados executa esse fluxo de forma transparente, sem que a aplicação precise se preocupar com a atualização do total — a trigger cuida de tudo automaticamente após cada inserção.

Padronização de Nomenclatura

Adotar uma convenção de nomenclatura clara para triggers é fundamental para a manutenibilidade e compreensão do seu banco de dados. Uma estrutura padronizada permite identificar rapidamente a função, o evento e a tabela associada a cada trigger.

A estrutura sugerida para triggers é: `trg_[evento]_[tabela_monitorada]_[acao]`

Por exemplo, a trigger que criamos no desafio, seguindo esta padronização, seria nomeada como:

```
trg_ai_ordem_servico_atualizar_valor_total_ordem
```

Neste nome temos: **trg** (trigger), **ai** (after insert), **ordem_servico** (tabela monitorada), **atualizar_valor_total_ordem** (ação de atualizar o valor total na tabela ordem).

---

ai = After Insert

au = After Update

ad = After Delete

bi = Before Insert

bu = Before Update

bd = Before Delete

Boas Práticas com Triggers

Triggers são ferramentas poderosas, mas exigem atenção. Seguir boas práticas garante que seu banco de dados permaneça performático, legível e fácil de manter.

1 Mantenha as triggers simples e com escopo único

Cada trigger deve ter responsabilidade clara e única. Triggers com múltiplas responsabilidades são difíceis de depurar e manter ao longo do tempo.

2 Documente o propósito de cada trigger

Use comentários SQL (`-- comentário`) para explicar por que a trigger existe. Isso facilita o trabalho de outros desenvolvedores que manterão o código.

3 Cuidado com triggers encadeadas (cascata)

Uma trigger pode disparar outra trigger. No SQLite, isso pode causar loops ou comportamentos inesperados — teste sempre o comportamento em cadeia.

4 Monitore o impacto em performance

Triggers adicionam processamento extra em cada operação de escrita. Em tabelas com alto volume de inserções, avalie se o impacto é aceitável para o seu cenário.

Quando Usar (e Quando Evitar) Triggers

✅ Use triggers quando...

- Precisar manter valores calculados automaticamente (totais, saldos, contadores)
- Quiser criar logs de auditoria de forma transparente
- Precisar replicar dados entre tabelas relacionadas ao salvar
- Quiser garantir integridade sem depender da camada de aplicação
- A regra de negócio for estável e bem definida

❌ Evite triggers quando...

- A lógica for muito complexa ou difícil de rastrear
- Houver alto volume de escrita e performance for crítica
- A regra de negócio mudar com frequência
- A mesma lógica já for tratada com segurança na aplicação
- O time não tiver familiaridade com triggers — invisibilidade pode gerar bugs

Criando a Trigger no Projeto da Assistência Técnica

Sempre que um serviço for inserido em uma ordem, o `valor_total` da ordem deve ser atualizado automaticamente.

1 Identifique as tabelas do seu projeto

Qual tabela recebe o serviço? Qual guarda o valor total? Qual coluna é a chave estrangeira que liga um serviço à sua ordem?

2 Escolha o evento e o momento

A trigger deve ser BEFORE ou AFTER? Ela deve disparar em INSERT, UPDATE ou DELETE? Justifique sua escolha.

3 Defina a regra de cálculo

Qual valor deve ser somado ao total? Como ele é obtido a partir da linha recém-inserida?

4 Escreva o CREATE TRIGGER

Monte a estrutura completa usando FOR EACH ROW, BEGIN...END e NEW para acessar a linha inserida. Teste a sintaxe no seu SGBD.

5 Teste o funcionamento

Insira um serviço na tabela e use SELECT para conferir se o `valor_total` da ordem mudou corretamente.

Dicas: CREATE TRIGGER · AFTER INSERT ON · FOR EACH ROW · NEW. · UPDATE ... SET ... WHERE

Desafio concluído quando o `valor_total` for atualizado sozinho após cada INSERT. Bom trabalho!

Anatomia da Trigger: Detalhe por Detalhe

Vamos desmembrar a trigger que criaremos para entender cada parte de sua construção e funcionamento.

`CREATE TRIGGER trg_ai_ordem_servico_atualizar_valor_total_ordem`

Inicia a criação da trigger e define seu nome, seguindo a padronização para indicar seu evento, tabela e ação.

`AFTER INSERT ON ordem_servico`

Especifica que a trigger será ativada **depois** (`AFTER`) de cada nova linha ser **inserida** (`INSERT`) na tabela `ordem_servico`.

`FOR EACH ROW`

Garante que o bloco de código da trigger será executado **uma vez para cada linha** que for inserida na tabela `ordem_servico`.

`BEGIN ... END;`

Delimita o bloco de comandos SQL que será executado quando a trigger for disparada. Tudo entre `BEGIN` e `END` é a ação da trigger.

`UPDATE ordem SET valor_total = valor_total + (NEW.preco * NEW.quantidade) WHERE id = NEW.id_ordem;`

Este é o coração da trigger: atualiza o `valor_total` na tabela `ordem`, somando o produto de `preco` e `quantidade` da linha **recém-inserida** (`NEW`) à ordem correspondente.

Anatomia da Trigger: Detalhe por Detalhe

Aqui está o código completo da trigger `trg_ai_ordem_servico_atualizar_valor_total_ordem` que implementa a lógica de acréscimo automático do `valor_total` na tabela `ordem`, sempre que um novo serviço é adicionado em `ordem_servico`.

```
CREATE TRIGGER trg_ai_ordem_servico_atualizar_valor_total_ordem
AFTER INSERT ON ordem_servico
FOR EACH ROW
BEGIN
  UPDATE ordem
  SET valor_total = valor_total + (NEW.preco * NEW.quantidade)
  WHERE id = NEW.id_ordem;
END;
```

Este bloco de código encapsula a regra de negócio, garantindo que o banco de dados mantenha a consistência dos dados de forma autônoma e transparente para a aplicação.

Além do INSERT: Exemplos de Triggers para UPDATE e DELETE

O mesmo padrão pode ser aplicado para outros eventos. Em um sistema de Ordem de Serviço completo, é necessário tratar também a **atualização** e a **exclusão** de serviços para manter o `valor_total` sempre correto.

Trigger para UPDATE

Quando um serviço é atualizado, remove o valor antigo e adiciona o novo:

```
CREATE TRIGGER atualizar_valor_total_update
AFTER UPDATE ON ordem_servico
FOR EACH ROW
BEGIN
  UPDATE ordem
  SET valor_total = valor_total
    - (OLD.preco * OLD.quantidade)
    + (NEW.preco * NEW.quantidade)
  WHERE id = NEW.id_ordem;
END;
```

Trigger para DELETE

Quando um serviço é removido, desconta o valor correspondente:

```
CREATE TRIGGER atualizar_valor_total_delete
AFTER DELETE ON ordem_servico
FOR EACH ROW
BEGIN
  UPDATE ordem
  SET valor_total = valor_total
    - (OLD.preco * OLD.quantidade)
  WHERE id = OLD.id_ordem;
END;
```

A palavra-chave `OLD` acessa os valores *anteriores* à operação, enquanto `NEW` acessa os valores *novos*. Em `DELETE`, apenas `OLD` está disponível.

Erros Comuns ao Usar Triggers

Conhecer os erros mais frequentes ajuda a evitar armadilhas que podem causar comportamentos inesperados ou degradar a performance do banco de dados.

Esquecer o `FOR EACH ROW`

No SQLite, o `FOR EACH ROW` é obrigatório. Sem ele, a trigger não compila corretamente. Sempre inclua essa cláusula em todas as suas triggers.

Não tratar UPDATE e DELETE

Criar apenas a trigger de `INSERT` e esquecer `UPDATE` e `DELETE` deixa o `valor_total` inconsistente quando serviços são editados ou removidos.

Criar loops acidentais

Se uma trigger faz um `UPDATE` na mesma tabela que a monitora, pode disparar a si mesma indefinidamente. Sempre verifique se há risco de recursão.

Trigger invisível para outros desenvolvedores

Triggers executam de forma silenciosa. Documente-as bem e inclua-as no controle de versão do projeto para que toda a equipe conheça sua existência.

Resumo: O que Aprendemos

🎯 Conceito

Triggers são blocos SQL executados automaticamente em resposta a eventos de `INSERT`, `UPDATE` ou `DELETE`.

🏗️ Cenário

Usamos um sistema de Ordem de Serviço com as tabelas `ordem` e `ordem_servico` para exemplificar o uso real.

⚙️ Lógica

A trigger calcula `NEW.preco * NEW.quantidade` e incrementa automaticamente o `valor_total` da ordem correspondente.

🔑 Palavras-chave

`NEW` acessa valores recém-inseridos; `OLD` acessa valores anteriores em `UPDATE` e `DELETE`.

✅ Boas Práticas

Mantenha triggers simples, documentadas, versionadas e sempre trate os três eventos: `INSERT`, `UPDATE` e `DELETE`.

Próximos Passos

Continue sua jornada com SQLite e automação de banco de dados

01

Pratique o exemplo

Crie as tabelas `ordem` e `ordem_servico` em um banco SQLite local e implemente as três triggers aprendidas hoje.

02

Explore triggers com BEFORE

Experimente triggers do tipo `BEFORE INSERT` para validar dados antes que sejam gravados, como verificar se o preço é maior que zero.

03

Aplique no projeto real

Identifique onde triggers poderiam substituir lógica repetitiva na aplicação e implemente a solução no banco de dados.

